

SQL SERVER 2022 • AVANZADO

Diagnóstico de queries lentas y timeouts en sistemas legacy (II)

Sesión 4 de 6 • De RBAR a conjuntos, CTEs y ventanas, tablas temporales, hints, índices y bloqueos

01

De RBAR a operaciones basadas en conjuntos

Row By Agonizing Row: por qué los cursores y los bucles son lentos y cómo pensar en conjuntos

Por qué el procesamiento fila a fila es lento

Cursor / WHILE

- Una sentencia por fila: compilación, bloqueos y log por iteración
- Cada iteración es una ida y vuelta al motor de ejecución
- Sin paralelismo ni modo batch
- Si hay COMMIT por fila: una espera WRITELOG por fila
- El optimizador solo ve fragmentos, nunca el problema completo

Operación de conjunto

- Una sentencia para todo el lote
- El optimizador elige joins, orden y paralelismo globalmente
- Lecturas secuenciales y read-ahead
- Un único registro de transacción agrupado
- Típicamente de 10 a 1.000 veces más rápido

Del cursor a una sola sentencia

```
-- ANTES (usp_RecalcularTotales): cursor sobre los pedidos pendientes
DECLARE c CURSOR FOR SELECT PedidoID FROM dbo.Pedidos WHERE Estado IN (1,2);
OPEN c; FETCH NEXT FROM c INTO @id;
WHILE @@FETCH_STATUS = 0 BEGIN
    SELECT @total = SUM(Cantidad * PrecioUnitario * (1 - Descuento/100))
    FROM dbo.LineasPedido WHERE PedidoID = @id;
    UPDATE dbo.Pedidos SET Total = ISNULL(@total, 0) WHERE PedidoID = @id;
    FETCH NEXT FROM c INTO @id;
END

-- DESPUÉS: una operación de conjunto
UPDATE p SET p.Total = ISNULL(l.Total, 0)
FROM dbo.Pedidos p
LEFT JOIN (SELECT PedidoID, SUM(Cantidad * PrecioUnitario * (1 - Descuento/100)) AS Total
           FROM dbo.LineasPedido GROUP BY PedidoID) l ON l.PedidoID = p.PedidoID
WHERE p.Estado IN (1,2) AND p.Total <> ISNULL(l.Total, 0);
```

El filtro final evita reescribir filas cuyo total ya es correcto: menos log, menos bloqueos y menos trabajo para el disparador.

Cuando el bucle es inevitable

- Procesar por lotes: TOP (5.000) en bucle, con clave de avance, no fila a fila
- Lotes por debajo de 5.000 filas evitan el escalado de bloqueos y limitan el tamaño de la transacción
- Si hay que usar cursor: LOCAL FAST_FORWARD (o READ_ONLY) evita el cursor dinámico por defecto
- Llamadas externas por fila (correo, API) no se resuelven en T-SQL: sacarlas a la aplicación
- Disparadores: siempre escritos para multifila (inserted/deleted como conjuntos)

En LegacyShop

trg_Pedidos_Auditoria asume una sola fila: `SELECT @id = PedidoID FROM inserted`. Con un UPDATE masivo registra solo una fila de auditoría. Es un fallo funcional, no solo de rendimiento.

02

CTEs, funciones de ventana y tablas temporales

Herramientas modernas de T-SQL y sus trampas de rendimiento

CTEs: legibilidad, no materialización

- Una CTE es una consulta con nombre: el optimizador la expande donde se usa, como una vista en línea
- No se materializa: si se referencia dos veces, se evalúa dos veces
- Para reutilizar un resultado costoso: tabla temporal con su índice
- CTE recursiva: jerarquías y grafos; MAXRECURSION 100 por defecto
- Ideal para encadenar pasos lógicos y para UPDATE/DELETE sobre ROW_NUMBER (deduplicar)

Mito frecuente

"He pasado la subconsulta a una CTE y va más rápido". Casi siempre es otro plan por otro motivo: estadísticas, caché o parámetros. Una CTE y una tabla derivada equivalentes producen el mismo plan.

Funciones de ventana: sustituir autojoins y subconsultas correladas

```
-- ANTES: subconsulta correlada por fila (acumulado del cliente)
SELECT p.PedidoID, p.FechaPedido, p.Total,
       (SELECT SUM(p2.Total) FROM dbo.Pedidos p2
        WHERE p2.ClienteID = p.ClienteID AND p2.FechaPedido <= p.FechaPedido) AS Acumulado
FROM dbo.Pedidos p WHERE p.ClienteID BETWEEN 100 AND 5000;

-- DESPUÉS: una pasada con ventana (ROWS explícito)
SELECT PedidoID, FechaPedido, Total,
       SUM(Total) OVER (PARTITION BY ClienteID ORDER BY FechaPedido, PedidoID
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS Acumulado,
       LAG(FechaPedido) OVER w AS PedidoAnterior
FROM dbo.Pedidos WHERE ClienteID BETWEEN 100 AND 5000
WINDOW w AS (PARTITION BY ClienteID ORDER BY FechaPedido, PedidoID); -- 2022
```

Sin marco explícito, ORDER BY implica RANGE: en modo fila usa un spool en disco. Con ROWS, en memoria.

Tablas temporales frente a variables de tabla

#TablaTemporal

- Estadísticas con histograma: buenas estimaciones
- Índices en la definición o posteriores
- Puede provocar recompilaciones (umbral de cambios)
- Inserciones en paralelo posibles
- Participa en la transacción: se deshace en ROLLBACK

@VariableTabla

- Sin estadísticas de columna; desde 2019 cuenta real de filas en la compilación diferida
- Solo índices en línea en la declaración
- No provoca recompilaciones por estadísticas
- La inserción en la variable es siempre en serie
- No se deshace en ROLLBACK: útil para registrar errores

03

Índices y hints con criterio

Índices cubrientes, filtrados y columnares; cuándo una sugerencia está justificada

Tres tipos de índice para sistemas legacy

1

Cubriente (INCLUDE)

Clave: columnas de igualdad primero, luego rango. INCLUDE: columnas del SELECT. Elimina los Key Lookups. Coste: más espacio y más escritura.

2

Filtrado (WHERE)

Indexa solo un subconjunto (Estado = pendiente). Pequeño y barato de mantener. Ojo: con parámetros no se usa salvo que el predicado coincida; exige opciones SET correctas.

3

Columnstore

Almacenamiento por columnas y modo batch. No agrupado para analítica sobre OLTP. En 2022: columnstore agrupado ordenado y eliminación de segmentos en cadenas.

Diseñar a partir de la evidencia

```
-- Sugerencias del optimizador (evidencia, no órdenes)
SELECT mid.statement, mid.equality_columns, mid.inequality_columns, mid.included_columns,
       migs.user_seeks, migs.avg_user_impact
FROM sys.dm_db_missing_index_details mid
JOIN sys.dm_db_missing_index_groups mig ON mig.index_handle = mid.index_handle
JOIN sys.dm_db_missing_index_group_stats migs ON migs.group_handle = mig.index_group_handle
WHERE mid.database_id = DB_ID() ORDER BY migs.user_seeks * migs.avg_user_impact DESC;

-- Índices que se escriben pero no se leen
SELECT OBJECT_NAME(i.object_id) AS tabla, i.name,
       us.user_seeks + us.user_scans + us.user_lookups AS lecturas, us.user_updates AS escrituras
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats us
      ON us.object_id = i.object_id AND us.index_id = i.index_id AND us.database_id = DB_ID()
WHERE OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1 AND i.index_id > 1
ORDER BY lecturas, escrituras DESC;
```

Ambas DMVs se reinician con la instancia: no borreís un índice "sin uso" tras dos días de actividad; pensad en el cierre de mes.

Hints: cuándo están justificados

- Una sugerencia congela una decisión: es buena hoy, con estos datos, y puede ser mala mañana
- Justificadas: RECOMPILE en consultas de informes poco frecuentes con parámetros muy variables
- Justificadas: MAXDOP en una consulta concreta, USE HINT documentados, FORCESEEK como parche temporal
- Peligrosas: NOLOCK por sistema (lecturas sucias, filas duplicadas u omitidas), LOOP/HASH JOIN, INDEX()
- Preferir Query Store hints: reversibles, visibles en un solo sitio y sin desplegar código

Sobre NOLOCK

NOLOCK no es "leer más rápido": es leer datos sin confirmar y aceptar que un scan concurrente con divisiones de página devuelva filas dos veces o se las salte. Si el problema son los bloqueos de lectura, la solución es RCSI.

Lab 6 · De RBAR a conjuntos

- Medir usp_RecalcularTotales (cursor) y reescribirlo como una sola sentencia
- Verificar que ambas versiones producen exactamente el mismo resultado
- Corregir el disparador de auditoría para que sea multifila
- Sustituir la subconsulta correlada por funciones de ventana
- Pasar el informe mensual de variable de tabla a tabla temporal y comparar planes

50 min

labs/
Lab06_RBAR_Conjuntos.md

Lab 7 · Índices y hints con criterio

- Convertir IX_Pedidos_ClienteID en cubriente y medir los dos clientes extremos
- Crear el índice de LineasPedido que falta y validar el efecto en todo el sistema
- Probar un índice filtrado con literal, con parámetro y con RECOMPILE
- Crear un columnstore no agrupado para el informe de ventas por categoría
- Consolidar las sugerencias de índices que faltan y detectar índices sin uso

45 min

labs/
Lab07_Indices_Hints.md

04

Bloqueos, escalado de bloqueos y deadlocks

Concurrencia: diagnosticar quién bloquea a quién y por qué

Bloqueos y escalado de bloqueos

- Granularidad: fila (KEY/RID), página, partición (HoBT) y tabla (OBJECT), con bloqueos de intención
- Escalado: al superar ≈ 5.000 bloqueos en un objeto con una sentencia, o por presión de memoria
- Escala a tabla (o a partición con `LOCK_ESCALATION = AUTO` en tablas particionadas)
- Si falla por incompatibilidad, reintenta cada 1.250 bloqueos adicionales
- Mitigar: lotes < 5.000 filas, índices que reduzcan filas tocadas, `LOCK_ESCALATION = DISABLE` con cuidado

Cómo se ve

Evento XE `lock_escalation`, esperas `LCK_M_*` de otras sesiones sobre `OBJECT`, y en `sys.dm_tran_locks` una sola fila de tipo `OBJECT` con modo `X` en lugar de miles de `KEY`.

Encontrar la cabeza de la cadena de bloqueos

```
SELECT r.session_id, r.blocking_session_id, r.wait_type, r.wait_time/1000 AS espera_s,  
       r.wait_resource, DB_NAME(r.database_id) AS bd,  
       SUBSTRING(st.text, r.statement_start_offset/2 + 1, 200) AS sentencia  
FROM sys.dm_exec_requests r  
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) st  
WHERE r.blocking_session_id <> 0;  
  
-- La cabeza: bloquea a otros pero no está bloqueada (puede estar "sleeping!")  
SELECT s.session_id, s.status, s.login_name, s.host_name, s.program_name,  
       s.open_transaction_count, s.last_request_end_time, st.text AS ultima_sentencia  
FROM sys.dm_exec_sessions s  
JOIN sys.dm_exec_connections c ON c.session_id = s.session_id  
CROSS APPLY sys.dm_exec_sql_text(c.most_recent_sql_handle) st  
WHERE s.session_id IN (SELECT blocking_session_id FROM sys.dm_exec_requests)  
AND s.session_id NOT IN (SELECT session_id FROM sys.dm_exec_requests WHERE blocking_session_id  
<> 0);
```

Para históricos: `sp_configure "blocked process threshold (s)" + evento XE blocked_process_report.`

Deadlocks: diagnóstico y patrones

- Ciclo de esperas: el monitor lo detecta (cada 5 s o menos) y elige víctima; error 1205
- La sesión `system_health` captura `xml_deadlock_report` sin configurar nada
- Patrón 1: dos procesos acceden a las mismas tablas en orden distinto
- Patrón 2: lookup deadlock, lector en índice no agrupado frente a escritor en el agrupado
- Patrón 3: escalado de bloqueos o scans por falta de índice que amplían la superficie de bloqueo

Remedios

Orden de acceso consistente, transacciones cortas, índices cubrientes, RCSI para eliminar conflictos lector-escritor y lógica de reintento en la aplicación para el error 1205.

Leer deadlocks de system_health y activar RCSI

```
-- Deadlocks recientes desde el fichero de system_health
SELECT x.value('@timestamp')[1], 'datetime2') AS momento, x.query('.') AS informe
FROM (SELECT CAST(event_data AS xml) AS e
      FROM sys.fn_xe_file_target_read_file('system_health*.xel', NULL, NULL, NULL)
      WHERE object_name = 'xml_deadlock_report') d
CROSS APPLY d.e.nodes('/event') AS t(x)
ORDER BY momento DESC;

-- Lectores que no bloquean a escritores (versionado en tempdb)
ALTER DATABASE LegacyShop SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK IMMEDIATE;
SELECT name, is_read_committed_snapshot_on FROM sys.databases;
```

RCSI cambia la semántica: un lector ve la última versión confirmada, no espera. Revisad lógica que dependa de leer y luego escribir.

Lab 8 · Bloqueos, escalado y deadlocks

- Provocar una cadena de bloqueos con una transacción huérfana y encontrar la cabeza
- Observar el escalado de bloqueos con sys.dm_tran_locks y XE lock_escalation
- Evitarlo con procesamiento por lotes
- Provocar un deadlock de orden de acceso y leerlo desde system_health
- Activar RCSI y repetir la prueba de bloqueo lector-escriptor

45 min

labs/
Lab08_Bloqueos_Deadlocks.m
d

Cierre de la sesión 4

- Pensar en conjuntos: la mayor ganancia de rendimiento con el menor coste de infraestructura
- Toda reescritura se valida contra el resultado original, incluidos los casos límite
- Índices a partir de evidencia; sugerencias solo justificadas y, mejor, como Query Store hints
- Bloqueos: buscar la cabeza (a menudo dormida); RCSI y lotes como primeras medidas
- Próxima sesión: seguridad avanzada y protección de datos en SQL Server 2022

Para casa

Buscad en vuestro código heredado cursores sin LOCAL FAST_FORWARD y disparadores que lean inserted en una variable escalar: `SELECT @x = ... FROM inserted.`